

General_and_cluster_permutation_PLE_slope_block

July 20, 2025

```
[1]: ##modules
    %%matplotlib widget
    %matplotlib inline
    #
    %%matplotlib qt
    import mne
    import numpy as np
    import matplotlib
    # Establecer un backend interactivo, como 'Qt5Agg', 'GTK3Agg', etc.
    # Esto depende de los backends disponibles en tu sistema.

    import matplotlib.pyplot as plt

    %matplotlib.use('TkAgg') # Asegúrate de que este backend está instalado.

    import pandas as pd
    import os
    import sys

    from mne.preprocessing import ICA, corrmmap, create_ecg_epochs, create_eog_epochs

    from os.path import join as pathjoin
    from time import time

    from pathlib import Path

    from autoreject import AutoReject

    # aplicar la acf EN epochs

    from statsmodels.tsa.stattools import acf

    from scipy.stats import permutation_test
    sys.path.append(r"G:\apuntes_mne\code_MOUS\scripts_functions_created")

    # Ahora importa la función
```

```
from print5 import print5
```

```
[2]: try:
    # Si se ejecuta como SCRIPT .py: usar __file__
    sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..
↪")))
except NameError:
    # Si se ejecuta como NOTEBOOK Jupyter: usar path relativo
    sys.path.append("..") # sube un nivel desde la carpeta actual del notebook

# --- Configuración dinámica de rutas ---
from get_paths_MOUS import get_paths_MOUS

# Parámetros editables
disco = "g"
modality = "visual"
layer_script = "block"
subj = "sub-V1001"
#subj = sys.argv[1] ## name of participant list

# Generar variables automáticamente
path_dict = get_paths_MOUS(disco=disco, modality=modality,
↪layer_script=layer_script, subj=subj)
globals().update(path_dict)

# Mostrar todos los paths generados
print("\n Rutas generadas:")
for k, v in path_dict.items():
    print(f"{k:<20} → {v}")
```

```
Rutas generadas:
datadir          → g:\MOUS_204\MOUS_visual
general_datadir  → g:\MOUS_204
output_preproc   → g:\MOUS_204\MOUS_visual\output_preproc
mri_dir          → g:\MOUS_204\sub-V1001\anat
meg_dir          → g:\MOUS_204\sub-V1001\meg
preproc_path     → g:\MOUS_204\MOUS_visual\output_preproc\preproc_block
channels_structure_path → g:\MOUS_204\channels_structure
epochs_path      →
g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\epochs_block
ICA_path         →
g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\ICA_block
```

```

epochs_clean_path      →
g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\epochs_clean_block
evoked_path            →
g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\evoked_block
source_path            → g:\MOUS_204\MOUS_visual\output_source\source_block
raw_hsp_path           →
g:\MOUS_204\MOUS_visual\output_source\source_block\raw_hsp
fwd_path               → g:\MOUS_204\MOUS_visual\output_source\source_block\fwd
inverse_path           →
g:\MOUS_204\MOUS_visual\output_source\source_block\inverse
output_analysis        → g:\MOUS_204\MOUS_visual\output_analysis
analysis_path          → g:\MOUS_204\MOUS_visual\output_analysis\analysis_block
ACW_path               →
g:\MOUS_204\MOUS_visual\output_analysis\analysis_block\acw_block
PLE_path               →
g:\MOUS_204\MOUS_visual\output_analysis\analysis_block\PLE_block
ISC_path               →
g:\MOUS_204\MOUS_visual\output_analysis\analysis_block\ISC_block
ISC_block_path         →
g:\MOUS_204\MOUS_visual\output_analysis\analysis_block\ISC_block

```

1 Tables and paths

1.0.1 Paths

ACW_path PLE_path

1.0.2 Tables and metrics

Block ACW f"acw_0_subjects_all_{layer_script}.pickle"

f"acw_50_subjects_all_{layer_script}.pickle"

f"autocorrelation_subjects_all_{layer_script}.pickle"

variables = acw_50_elect_all_epoch_all // 'intercept_y_elect_all_epoch_all',
'slope_elect_all_epoch_all'

PLE f"table_PLE_slope_intercept_subjects_all_{layer_script}.pickle"

f"table_PLE_subjects_all_{layer_script}.pickle"

dynamic f"dynamic_autocorrelation_results_subjects_all_{layer_script}.pickle"

f"dynamic_autocorrelation_subjects_all_{layer_script}.pickle"

variables = acw_50_slope_elect_all_epoch_all // acw_50_std_elect_all_epoch_all

[3]: *## Variables of script*

```

path = PLE_path
name_table = f"table_PLE_slope_intercept_subjects_all_{layer_script}.pickle"
table_df = pd.read_pickle(f"{path}\\{name_table}")

#METRIC NAMES
metric='slope_elect_all_epoch_all'

if '_elect_all_epoch_all' in metric:
    print("_elect_all_epoch_all in metric")
    metric_name = metric.replace('_elect_all_epoch_all', "")
else:
    metric_name = metric

##CONDITION NAMES
condition_1_name="zinnen"
condition_2_name="woorden"

condition_names = [condition_1_name, condition_2_name]

#TYPE OF DIFFERENCES
difference = "inverse" # "normal" or "inverse"

## this will be used in permutation differences
if difference == "normal":
    alternative = "greater"
    # this is for cluster permutation test
    tail=1
    threshold_direction =1
else:
    alternative="less"
    tail=-1
    threshold_direction =-1

# ##creacion dataframe acw_0

# acw_50_df = acw_all[["Subject", "Condition", "Epoch",
#                      "Elect", "acw_50_elect_all_epoch_all"]]
# acw_50_df.to_pickle(ACW_path / f"acw_50_df.pickle")

# del acw_50_df
# acw_0_df = acw_all[["Subject", "Condition", "Epoch",
#                      "Elect", "metric_elect_all_epoch_all"]]
# acw_0_df.to_pickle(ACW_path / f"acw_0_df.pickle")

# del acw_all

```

```
# del acw_0_df
```

```
_elect_all_epoch_all in metric
```

```
[4]: channels = pd.read_csv(channels_structure_path / f"channels_mag_{modality}.csv")
channels_mag=channels[channels[f"canal_efectivo_{modality}"].
↳notna()][f"canal_efectivo_{modality}"]

channels_mag=channels_mag.tolist()
print5(channels_mag)
indice_channels_efectivos = channels[channels[f"canal_efectivo_{modality}"].
↳notna()]["indice"]
indice_channels_efectivos=indice_channels_efectivos.tolist()
# del channels

##valores de las columnas
condition = table_df["Condition"].unique()
print5("Condiciones en los datos:", condition)

subjects = table_df["Subject"].unique()
print5("Sujetos en los datos:", subjects)

elect_all= table_df["Elect"].unique()
print5("sensores en los datos:", elect_all)

epochs_all= table_df["Epoch"].unique()
print5("Epochs en los datos:", epochs_all)
## get info

#read epochs to build evoked
epochs_zinnen=mne.read_epochs(epochs_clean_path /
↳f"{subj}_epochs_zinnen_{layer_script}-epo.fif")
len(epochs_zinnen.pick("mag", exclude="bads").ch_names)

evokeds_zinnen=epochs_zinnen.average()
## evokeds_zinnen es una lista

##cojo el primer elemento, solo tengo una lista
evoked_zinnen=evokeds_zinnen

info=evoked_zinnen.info
del epochs_zinnen
del evokeds_zinnen
del evoked_zinnen
```

```
['MLC11-4304', 'MLC12-4304', 'MLC13-4304', 'MLC14-4304', 'MLC15-4304',
'MLC16-4304', 'MLC17-4304', 'MLC21-4304', 'MLC22-4304', 'MLC23-4304',
'MLC24-4304', 'MLC25-4304', 'MLC31-4304', 'MLC32-4304', 'MLC41-4304',
```


'MRT51-4304', 'MRT52-4304', 'MRT53-4304', 'MRT54-4304', 'MRT55-4304',
 'MRT56-4304', 'MRT57-4304', 'MZC01-4304', 'MZC02-4304', 'MZC03-4304',
 'MZC04-4304', 'MZFO1-4304', 'MZFO2-4304', 'MZFO3-4304', 'MZ001-4304',
 'MZ002-4304', 'MZ003-4304', 'MZP01-4304']
 Condiciones en los datos: ['zinnen' 'woorden']
 Sujetos en los datos: ['sub-V1001' 'sub-V1002' 'sub-V1005' 'sub-V1007'
 'sub-V1008' 'sub-V1009'
 'sub-V1010' 'sub-V1011' 'sub-V1012' 'sub-V1013' 'sub-V1016' 'sub-V1019'
 'sub-V1020' 'sub-V1022' 'sub-V1024' 'sub-V1029' 'sub-V1030' 'sub-V1031']
 sensores en los datos: ['MLC11-4304' 'MLC12-4304' 'MLC13-4304' 'MLC14-4304'
 'MLC15-4304'
 'MLC16-4304' 'MLC17-4304' 'MLC21-4304' 'MLC22-4304' 'MLC23-4304'
 'MLC24-4304' 'MLC25-4304' 'MLC31-4304' 'MLC32-4304' 'MLC41-4304'
 'MLC42-4304' 'MLC51-4304' 'MLC52-4304' 'MLC53-4304' 'MLC54-4304'
 'MLC55-4304' 'MLC61-4304' 'MLC62-4304' 'MLC63-4304' 'MLF11-4304'
 'MLF12-4304' 'MLF13-4304' 'MLF14-4304' 'MLF21-4304' 'MLF22-4304'
 'MLF23-4304' 'MLF24-4304' 'MLF25-4304' 'MLF31-4304' 'MLF32-4304'
 'MLF33-4304' 'MLF34-4304' 'MLF35-4304' 'MLF41-4304' 'MLF42-4304'
 'MLF43-4304' 'MLF44-4304' 'MLF45-4304' 'MLF46-4304' 'MLF51-4304'
 'MLF52-4304' 'MLF53-4304' 'MLF54-4304' 'MLF55-4304' 'MLF56-4304'
 'MLF61-4304' 'MLF63-4304' 'MLF64-4304' 'MLF65-4304' 'MLF66-4304'
 'MLF67-4304' 'ML011-4304' 'ML012-4304' 'ML013-4304' 'ML014-4304'
 'ML021-4304' 'ML022-4304' 'ML023-4304' 'ML024-4304' 'ML031-4304'
 'ML032-4304' 'ML033-4304' 'ML034-4304' 'ML041-4304' 'ML042-4304'
 'ML043-4304' 'ML044-4304' 'ML051-4304' 'ML052-4304' 'ML053-4304'
 'MLP11-4304' 'MLP12-4304' 'MLP21-4304' 'MLP22-4304' 'MLP23-4304'
 'MLP31-4304' 'MLP32-4304' 'MLP33-4304' 'MLP34-4304' 'MLP35-4304'
 'MLP41-4304' 'MLP42-4304' 'MLP43-4304' 'MLP44-4304' 'MLP45-4304'
 'MLP51-4304' 'MLP52-4304' 'MLP53-4304' 'MLP54-4304' 'MLP55-4304'
 'MLP56-4304' 'MLP57-4304' 'MLT11-4304' 'MLT12-4304' 'MLT13-4304'
 'MLT14-4304' 'MLT15-4304' 'MLT16-4304' 'MLT21-4304' 'MLT22-4304'
 'MLT23-4304' 'MLT24-4304' 'MLT25-4304' 'MLT26-4304' 'MLT27-4304'
 'MLT31-4304' 'MLT32-4304' 'MLT33-4304' 'MLT34-4304' 'MLT35-4304'
 'MLT36-4304' 'MLT41-4304' 'MLT42-4304' 'MLT43-4304' 'MLT44-4304'
 'MLT45-4304' 'MLT46-4304' 'MLT47-4304' 'MLT51-4304' 'MLT52-4304'
 'MLT53-4304' 'MLT54-4304' 'MLT55-4304' 'MLT56-4304' 'MLT57-4304'
 'MRC11-4304' 'MRC12-4304' 'MRC13-4304' 'MRC14-4304' 'MRC15-4304'
 'MRC16-4304' 'MRC17-4304' 'MRC21-4304' 'MRC22-4304' 'MRC23-4304'
 'MRC24-4304' 'MRC25-4304' 'MRC31-4304' 'MRC32-4304' 'MRC41-4304'
 'MRC42-4304' 'MRC51-4304' 'MRC52-4304' 'MRC53-4304' 'MRC54-4304'
 'MRC55-4304' 'MRC61-4304' 'MRC62-4304' 'MRC63-4304' 'MRF11-4304'
 'MRF12-4304' 'MRF13-4304' 'MRF14-4304' 'MRF21-4304' 'MRF22-4304'
 'MRF23-4304' 'MRF24-4304' 'MRF25-4304' 'MRF31-4304' 'MRF32-4304'
 'MRF33-4304' 'MRF34-4304' 'MRF35-4304' 'MRF41-4304' 'MRF42-4304'
 'MRF43-4304' 'MRF44-4304' 'MRF45-4304' 'MRF46-4304' 'MRF51-4304'
 'MRF52-4304' 'MRF53-4304' 'MRF54-4304' 'MRF55-4304' 'MRF56-4304'
 'MRF61-4304' 'MRF62-4304' 'MRF63-4304' 'MRF64-4304' 'MRF65-4304'
 'MRF66-4304' 'MRF67-4304' 'MR011-4304' 'MR012-4304' 'MR013-4304'

```
'MR014-4304' 'MR021-4304' 'MR022-4304' 'MR023-4304' 'MR024-4304'
'MR031-4304' 'MR032-4304' 'MR033-4304' 'MR034-4304' 'MR041-4304'
'MR042-4304' 'MR043-4304' 'MR044-4304' 'MR051-4304' 'MR052-4304'
'MR053-4304' 'MRP11-4304' 'MRP12-4304' 'MRP21-4304' 'MRP22-4304'
'MRP23-4304' 'MRP31-4304' 'MRP32-4304' 'MRP33-4304' 'MRP34-4304'
'MRP35-4304' 'MRP41-4304' 'MRP42-4304' 'MRP43-4304' 'MRP44-4304'
'MRP45-4304' 'MRP51-4304' 'MRP52-4304' 'MRP53-4304' 'MRP54-4304'
'MRP55-4304' 'MRP56-4304' 'MRP57-4304' 'MRT11-4304' 'MRT12-4304'
'MRT13-4304' 'MRT14-4304' 'MRT15-4304' 'MRT16-4304' 'MRT21-4304'
'MRT22-4304' 'MRT23-4304' 'MRT24-4304' 'MRT25-4304' 'MRT26-4304'
'MRT27-4304' 'MRT31-4304' 'MRT32-4304' 'MRT33-4304' 'MRT34-4304'
'MRT35-4304' 'MRT36-4304' 'MRT37-4304' 'MRT41-4304' 'MRT42-4304'
'MRT43-4304' 'MRT44-4304' 'MRT45-4304' 'MRT46-4304' 'MRT47-4304'
'MRT51-4304' 'MRT52-4304' 'MRT53-4304' 'MRT54-4304' 'MRT55-4304'
'MRT56-4304' 'MRT57-4304' 'MZC01-4304' 'MZC02-4304' 'MZC03-4304'
'MZC04-4304' 'MZF01-4304' 'MZF02-4304' 'MZF03-4304' 'MZ001-4304'
'MZ002-4304' 'MZ003-4304' 'MZP01-4304']
```

Epochs en los datos: [0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
19 20 21 22 23]

Reading g:\MOUS_204\MOUS_visual\output_preproc\preproc_block\epochs_clean_block\
sub-V1001_epochs_zinnen_block-epo.fif ...

Found the data of interest:

t = 0.00 ... 60000.00 ms

0 CTF compensation matrices available

Not setting metadata

24 matching events found

No baseline correction applied

0 projection items activated

1.1 Selection of values

```
[5]: ### testing with pandas, nomenclature includes ONLY what is preserved, not what
      ↳ has been averaged

      # this is for comparing between conditions, as it has dimension=number of
      ↳ subjects
metric_cond_subj_epoch_mean_elect_mean= table_df.groupby(["Subject",
      ↳ "Condition"])[metric].mean().reset_index()

      #this is for plotting results (plot_topomap) as it has dimension=number of
      ↳ electrodes
metric_cond_subj_mean_epoch_mean_elect=table_df.groupby(["Condition",
      ↳ "Elect"])[metric].mean().reset_index()

      #this is for cluster permutation test, as it has dimension=number of subjects
      ↳ and electrodes
```

```
metric_cond_subj_epoch_mean_elect=table_df.groupby(["Subject", "Condition",
↳ "Elect"])[metric].mean().reset_index()
```

2 Plots

2.1 Plot topomat of activation

```
[6]: #
for exp_condition in condition_names:
    # Filtrar el DataFrame por la condición actual
    filtered_df = metric_cond_subj_mean_epoch_mean_elect.query("Condition ==
↳ @exp_condition")

    # Extraer los valores de la métrica y los nombres de los electrodos
    values = filtered_df[metric].values.flatten()
    electrodes = filtered_df["Elect"].values

    # Asegurarse de que los valores y los electrodos tienen la misma longitud
    if len(values) != len(electrodes):
        raise ValueError("Los valores y los electrodos deben tener la misma
↳ longitud.")

    # Crea figura + eje manual
    fig, ax = plt.subplots(figsize=(6, 6))

    # Imprimir para depuración
    # print5(f"Valores para {exp_condition}: {values}")
    # print5(f"Electrodos para {exp_condition}: {electrodes}")

    # Dibujar topomap DENTRO de ese eje
    im, _ = mne.viz.plot_topomap(
        data=values,
        pos=info,
        axes=ax,          # + esto asegura que se dibuja donde tú quieres
        cmap='RdBu_r',
        vlim=(-np.max(np.abs(values)), np.max(np.abs(values))),
        mask=None,
        contours=0,
        show=False        # + evita que se muestre automáticamente
    )

    # Título
    fig.suptitle(f"{metric_name} values in {exp_condition}", fontsize=20)

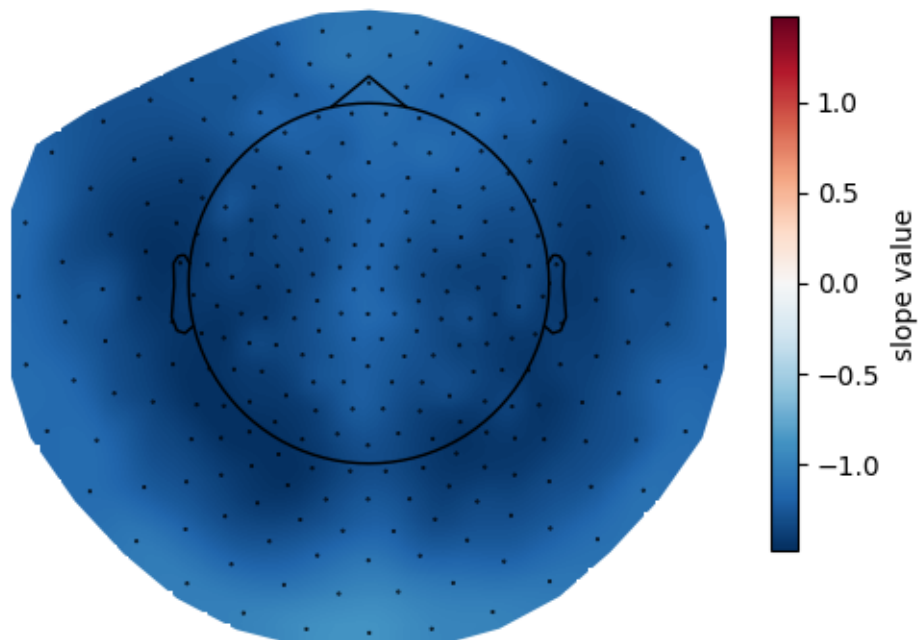
    # Añadir barra de color (leyenda)
    cbar = fig.colorbar(im, ax=ax, shrink=0.6)
```

```
cbar.set_label(f"{metric_name} value")
```

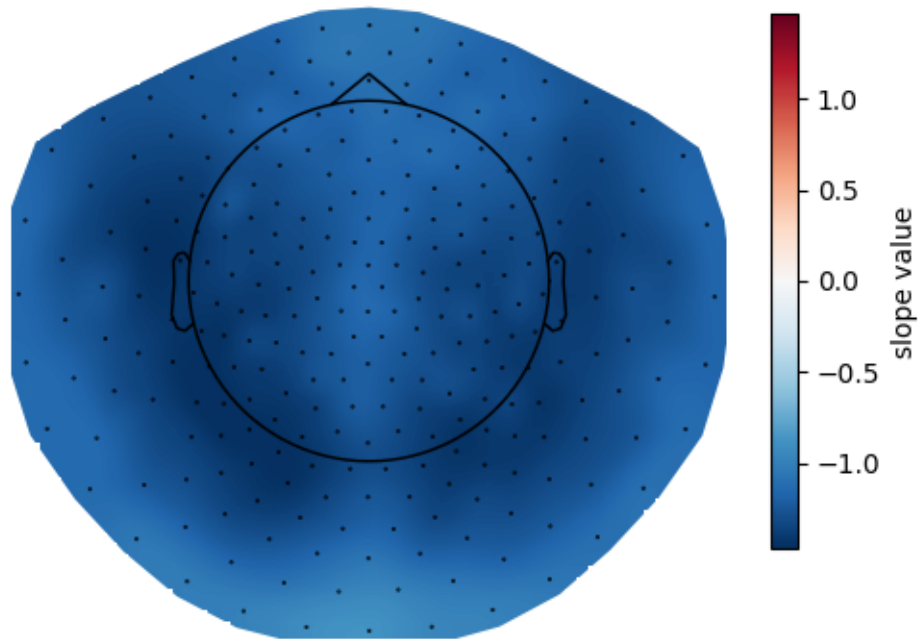
```
# Mostrar figura
```

```
plt.show()
```

slope values in zinnen



slope values in woorden



2.1.1 linear plots

```
[7]: from scipy.stats import gaussian_kde
import numpy as np
import matplotlib.pyplot as plt

#metric_cond_subj_epoch_mean_elect shouldnt be used but plot gets
↳uninformative, when all subjects, use metric_cond_subj_epoch_mean_elect_mean

# Crear diccionario con los arrays planos de cada condición
dict_conditions_flat = {}
for cond in condition_names:
    condition_flat = metric_cond_subj_epoch_mean_elect.query("Condition ==
↳@cond")[metric].values.flatten()
    dict_conditions_flat[cond] = condition_flat
```

```

# Crear figura
plt.figure(figsize=(10, 5))

# Colores para condiciones (puedes ajustar o automatizar más)
colors = ['blue', 'red', 'green', 'orange', 'purple']
counts_dict = {}
kde_dict = {}

for idx, cond in enumerate(condition_names):
    data = dict_conditions_flat[cond]

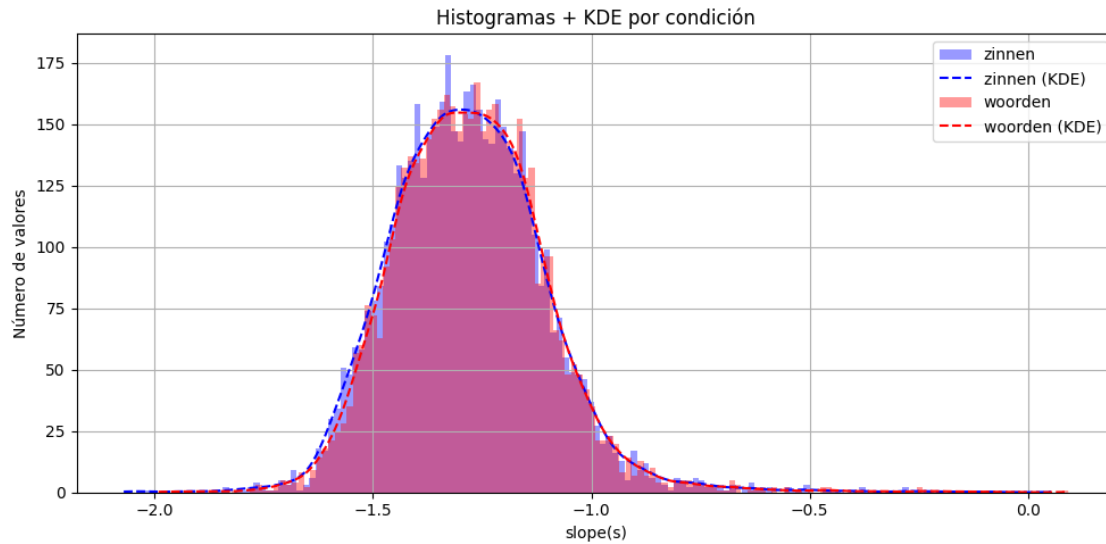
    # Histograma
    counts, bins, _ = plt.hist(
        data,
        bins=150,
        alpha=0.4,
        label=f'{cond}',
        color=colors[idx % len(colors)]
    )
    counts_dict[cond] = counts

    # KDE
    kde = gaussian_kde(data)
    x_vals = np.linspace(data.min(), data.max(), 300)
    scaled_kde = kde(x_vals) * len(data) * np.diff(bins)[0]
    kde_dict[cond] = scaled_kde

    # Graficar KDE
    plt.plot(
        x_vals,
        scaled_kde,
        linestyle='--',
        color=colors[idx % len(colors)],
        label=f'{cond} (KDE)'
    )

# Ejes y leyenda
plt.xlabel(f"{metric_name}(s)")
plt.ylabel("Número de valores")
plt.title("Histogramas + KDE por condición")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```



3 General differences between woorden and zinnen conditions

```
[8]: #take values for electrodes, dimension=number of subjects
dict_conditions_flat = {}
for cond in condition_names:
    condition_flat = metric_cond_subj_epoch_mean_elect_mean.query("Condition ==_
↪@cond") [metric].values.flatten()
    dict_conditions_flat[cond] = condition_flat

##dependent condition
def paired_statistic(diff, _):
    return np.mean(diff)

print5(f"Differences in {metric_name} between conditions")
for cond in condition_names:
    if cond==condition_1_name:
        print(f"x is {cond} and {condition_1_name}")
        x= dict_conditions_flat[cond]
    if cond==condition_2_name:
        print(f"y is {cond} and {condition_2_name}")
        y=dict_conditions_flat[cond]

diff = x - y
# Permutation test
res = permutation_test(
    (diff, np.zeros_like(diff)),
    statistic=paired_statistic,
```

```

        vectorized=False,
        n_resamples=10000,
        alternative=alternative,
        random_state=42
    )

    print5(f"Statistic value: {res.statistic}")
    print5(f"p-value: {res.pvalue}")

    # d of Cohen

    mean_diff = np.mean(diff)
    std_diff = np.std(diff, ddof=1) # ddof=1 para usar la desviación muestral

    cohens_d = mean_diff / std_diff

    print5(f"Cohen's d (paired): {cohens_d:.3f}")

```

Differences in slope between conditions

x is zinnen and zinnen

y is woorden and woorden

Statistic value: -0.00725

p-value: 0.00660

Cohen's d (paired): -0.60000

4 Cluster analysis

4.1 Datos a comparar

ahora tengo por cada condición: subject x epoch x channel

procedimiento

- promediar a nivel de epoca para tener subject x channel (luego probaré a hacerlo de otra manera sin promediar)
- ejecutar permutation cluster based analysis usando mi acw_zinnen vs mi acw_woorden que va a estar como subject x channel
 - el cluster solo puede ser espacial, porque mis epocas son discontinuas, creo que esto es adjacency

Resultados T_obs: La estadística observada en cada punto. clusters: Los clústeres formados por puntos significativos adyacentes. cluster_p_values: Los valores p de cada clúster (ya corregidos por comparaciones múltiples). H0: La distribución nula obtenida por permutaciones.

Supongo que clusters y sus valores de significación Luego no se como se compara entre condiciones, pero de momemento vamos así

4.2 Cluster in differences

```
[9]: ##get adjacency value
adjacency_reduced = pd.read_pickle(channels_structure_path /
    ↪f"adjacency_reduced_{modality}.pkl")
###print5(f"shape adjacency_reduced: {adjacency_reduced.shape}")

[10]: ##note that condition_1 and condition_2 are 2D arrays with shape (n_subjects,
    ↪n_channels)

#here im using various things
##.query("Condition ==@cond") =
    ↪[metric_cond_subj_epoch_mean_elect["Condition"]=cond]
#pivot index="Subject",columns="Elect", values=metric) gives toy a a table with
    ↪indexes and columns
#to_numpy converts the table to a numpy array, removin names of indexes and
    ↪columns

for cond in condition_names:
    if cond==condition_1_name:
        print(f"x is {cond} and {condition_1_name}")
        x= metric_cond_subj_epoch_mean_elect.query("Condition ==@cond").
    ↪pivot(index="Subject",columns="Elect", values=metric).to_numpy()
    if cond==condition_2_name:
        print(f"y is {cond} and {condition_2_name}")
        y=metric_cond_subj_epoch_mean_elect.query("Condition ==@cond").
    ↪pivot(index="Subject",columns="Elect", values=metric).to_numpy()

print(f"x.shape is {x.shape}")
print(f"y.shape is {y.shape}")
diff= x-y
print5(f"diff.shape is {diff.shape}")

# as I'm going to apply differences to same subjects, i need a DEPENDENT t test
    ↪kind of thing,
# so instead of mne.stats.permutation_cluster_test I'm going to use mne.stats.
    ↪permutation_cluster_1samp_test

# mne.stats.permutation_cluster_1samp_test(X, threshold=None,
    ↪n_permutations=1024, tail=0, stat_fun=None, adjacency=None, n_jobs=None,
    ↪seed=None, max_step=1, exclude=None,
#
    ↪step_down_p=0, t_power=1,
    ↪out_type='indices', check_disjoint=False, buffer_size=1000, verbose=None)
```

```

# as im going to put zinnen before woorden i need tail=1

#instead of puting X with bot condigions, i will put the difference between
↳them diff=zinnen-woorden
# i modify the value of the tail to be -1, because i want to see if the values
↳of zinnen are lower than the values of woorden
from scipy.stats import t
p_cluster_value = 0.001
t_thresh = (threshold_direction ) * t.ppf(1 - p_cluster_value, df=len(subjects)
↳- 1)

t_obs_diff, clusters_diff, clusters_pv_diff, HO=mne.stats.
↳permutation_cluster_1samp_test(diff, threshold=t_thresh,
↳n_permutations=1024, tail=tail,

↳stat_fun=None, adjacency=adjacency_reduced, n_jobs=15, seed=None,
↳max_step=1, exclude=None,

↳step_down_p=0, t_power=1, out_type='indices', check_disjoint=False,
↳buffer_size=1000, verbose=None)

# Obtenemos los clusters como listas de índices
# Filtrar clusters con p < 0.05
#zip links clusters y p-values
significant_clusters_diff = [
    cluster for cluster, p in zip(clusters_diff, clusters_pv_diff) if p < 0.05
]

print5(f"{len(significant_clusters_diff)} significative clusters found.")
print5(f"{metric_name}Clusters significative: {significant_clusters_diff}")

```

x is zinnen and zinnen

y is woorden and woorden

x.shape is (18, 273)

y.shape is (18, 273)

diff.shape is (18, 273)

stat_fun(H1): min=-4.667770657680899 max=0.6795120813074376

Running initial clustering ...

Found 1 cluster

c:\Users\UCM\anaconda3\envs\env_meg\Lib\site-packages\tqdm\auto.py:21:

TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html

from .autonotebook import tqdm as notebook_tqdm

100%| | Permuting : 1023/1023 [00:03<00:00, 297.88it/s]

```

1 significant clusters found.
slopeClusters significative: [(array([135, 136, 140, 141, 142, 143, 145, 149,
211, 212, 215, 216, 217, 218, 220, 222, 223, 224]),)]

```

```

[11]: ###PLOTSS

mask_diff = np.zeros(t_obs_diff.shape, dtype=bool)

##significant_clusters_diff is a list of tuples of arrays
# cluster[0] is the electrodes of the cluster
for cluster in significant_clusters_diff:
    mask_diff[cluster[0]] = True

##in data i use the value of t_obs_diff
#pos takes the information from evoked.info object
#mask takes the mask_diff object
#in t stat i don't change it because the p value will be calculated with
    ↳permutations, so no problem when using a parametric test,
# even if the distribution is non parametric

# Crear figura y eje
fig, ax = plt.subplots(figsize=(6, 6))

# Dibujar el topomap
im, _ = mne.viz.plot_topomap(
    data=t_obs_diff,
    pos=info,
    mask=mask_diff,
    axes=ax,
    cmap='RdBu_r',
    vlim=(-np.max(np.abs(t_obs_diff)), np.max(np.abs(t_obs_diff))),
    mask_params=dict(marker='o', markerfacecolor='yellow', markersize=7),
    contours=0,
    show=False
)

# Añadir título
fig.suptitle(f"{metric_name} Clusters in differences,
    ↳p_threshold={p_cluster_value}", fontsize=14)

# Añadir barra de color
cbar = fig.colorbar(im, ax=ax, shrink=0.6)
cbar.set_label("T-statistic value")

```

```
# Mostrar el gráfico  
plt.show()
```

slope Clusters in differences, $p_threshold=0.001$

